Universidad Nacional de Colombia

Facultad de Ingeniería

Estructuras de Datos

Implementación y Análisis de Complejidad de Listas, Pilas y Colas en Java

Informe de proyecto

Integrantes:
Egovail Cardozo Juan Daniel
Romero Villalba Jean Pierre
Toro Moreno Kevin Andrés

Profesor: David Alberto Herrera Álvarez
Periodo: 2026-1

Bogotá, Colombia

22 de marzo de 2026

Índice

1. Objetivo **3**

2. Descripción general del problema **3**

3. Explicación de la implementación **3**

3.1. Implementación de `List` 3

3.2. Implementación de `MyStack<T>` 6

3.3. Implementación de `MyQueue<T>` 8

4. Métodos implementados **10**

4.1. Métodos de `List` 10

4.2. Métodos de `MyStack<T>` 12

4.3. Métodos de `MyQueue<T>` 13

5. Análisis de complejidad **14**

5.1. Análisis teórico 14

5.1.1. Complejidad teórica de `ListADT<T>` 14

5.1.2. Complejidad teórica de `MyStack<T>` 16

5.1.3. Complejidad teórica de `MyQueue<T>` 17

5.2. Metodología experimental 18

5.2.1. Estrategia general de medición 19

5.2.2. Ejecución de múltiples repeticiones 19

5.2.3. Configuración de las corridas 20

5.2.4. Conjunto de tamaños de entrada 20

5.2.5. Criterio de construcción de las pruebas 21

5.2.6. Consideraciones sobre operaciones compuestas 21

5.2.7. Registro de resultados 21

5.2.8. Generación de gráficas 21

5.2.9. Limitaciones metodológicas 22

5.3. Análisis empírico 22

5.3.1. Análisis empírico de `MyStack<T>` 23

5.3.2. Análisis empírico de `MyQueue<T>` 24

5.3.3. Comparación entre `MyStack<T>` y `MyQueue<T>` 25

5.3.4. Análisis empírico de `List` 29

5.3.5. Comparativa entre implementaciones de `List` 31

5.3.6. Comparación entre estructuras para operaciones equivalentes 35

6. Conclusiones **37**

7. Repositorio **38**

1. Objetivo

Desarrollar e implementar distintas variantes de la estructura de datos `ListADT<T>` mediante listas enlazadas, así como las estructuras `MyStack<T>` y `MyQueue<T>` utilizando arreglos dinámicos.

Adicionalmente, analizar la complejidad de sus métodos principales tanto desde un enfoque teórico como experimental, evaluando el comportamiento de cada implementación mediante mediciones de tiempo y comparaciones gráficas.

Finalmente, establecer relaciones entre métodos equivalentes de las diferentes estructuras, identificando las implementaciones más eficientes según su desempeño observado y su complejidad esperada.

2. Descripción general del problema

Se desarrollaron:

- Cuatro variantes de `ListADT<T>` usando listas enlazadas:
  - Simplemente enlazada sin cola
  - Simplemente enlazada con cola
  - Doblemente enlazada sin cola
  - Doblemente enlazada con cola
- `MyStack<T>` usando arreglo dinámico
- `MyQueue<T>` usando arreglo dinámico

Posteriormente se realizó un análisis teórico y empírico de los métodos implementados.

3. Explicación de la implementación

3.1. Implementación de `List`

La implementación de la estructura `List` se organizó a partir de un contrato general definido mediante la interfaz `ListADT<T>`, la cual establece las operaciones fundamentales que deben soportar todas las variantes de listas desarrolladas en el proyecto, incluyendo inserciones, eliminaciones, consultas y búsqueda de elementos.

Listing 1: Contrato general definido en `ListADT<T>`

```
public interface ListADT<T>{
    void pushFront(T value);
    void pushBack(T value);
    T popFront();
    T popBack();
    boolean isEmpty();
    T topFront();
    T topBack();
    int size();
    Position<T> find(T value);
    void erase(Position<T> position);
    void addBefore(Position<T> position, T value);
    void addAfter(Position<T> position, T value);
}
```

Esta interfaz permite desacoplar la definición del comportamiento de la estructura de su implementación concreta, garantizando uniformidad entre las distintas variantes de listas enlazadas.

Una decisión de diseño clave fue la introducción de la interfaz `Position<T>`, la cual representa una abstracción de posición dentro de la lista. Esta interfaz permite que el método `find` retorne una referencia lógica al nodo, en lugar de retornar únicamente el valor.

Listing 2: Abstracción de posición usada en la lista

```
public interface Position<T>{
    T getValue();
}
```

Gracias a esta abstracción, operaciones como `erase`, `addBefore` y `addAfter` pueden trabajar directamente sobre una posición previamente obtenida, evitando ambigüedades cuando existen elementos repetidos y separando la lógica de búsqueda de la manipulación de la estructura.

Cada implementación concreta define una clase interna `Node<T>`, la cual encapsula la representación real del nodo y, al mismo tiempo, implementa la interfaz `Position<T>`.

Listing 3: Nodo interno que implementa `Position<T>`

```java
private static class Node<T> implements Position<T>{
    private T value;
    private Node<T> next;

    public Node(T value){
        this.value = value;
    }

    @Override
    public T getValue() {
        return value;
    }
}
```

Este diseño permite retornar nodos como posiciones sin exponer directamente la estructura interna de la lista, manteniendo así un mayor nivel de encapsulamiento.

A partir de esta base común, se desarrollaron cuatro variantes de listas enlazadas:

- Lista simplemente enlazada sin cola
- Lista simplemente enlazada con cola
- Lista doblemente enlazada sin cola
- Lista doblemente enlazada con cola

Las diferencias entre estas implementaciones radican principalmente en dos decisiones estructurales: la presencia o ausencia de enlaces dobles (`prev`) y el uso de una referencia al último nodo (`tail`). Estas decisiones afectan directamente la forma en que se actualizan las referencias entre nodos y, por ende, la eficiencia de las operaciones.

Por ejemplo, en listas simplemente enlazadas, la ausencia de una referencia al nodo anterior implica que operaciones como `erase` o `addBefore` requieren recorrer la lista para encontrar el predecesor del nodo objetivo. En contraste, en listas doblemente enlazadas, este acceso está disponible directamente mediante la referencia `prev`, lo que permite realizar dichas operaciones de manera más eficiente.

De manera similar, la incorporación de una referencia `tail` elimina la necesidad de recorrer la estructura para acceder al último nodo, optimizando operaciones como `pushBack` y `topBack`. Sin embargo, en listas simplemente enlazadas con cola, la eliminación del último elemento sigue requiriendo un recorrido, ya que no se dispone de acceso directo al nodo anterior.

Adicionalmente, todas las implementaciones mantienen un atributo `size`, el cual se actualiza en cada inserción y eliminación, permitiendo que la operación `size()` se ejecute en tiempo constante.

En conjunto, estas decisiones de diseño permiten construir una familia de estructuras con el mismo comportamiento externo, pero con diferencias significativas en su desempeño interno. Estas diferencias serán analizadas en detalle en la siguiente sección, donde se estudia la complejidad temporal de cada operación en función de la implementación utilizada.

3.2. Implementación de `MyStack<T>`

La estructura `MyStack<T>` fue definida mediante una interfaz que establece el comportamiento fundamental de una pila, siguiendo el principio LIFO (*Last In, First Out*). Esta interfaz permite separar el contrato de la estructura de su implementación concreta.

Listing 4: Contrato definido en `MyStack<T>`

```
public interface MyStack<T> {
    void push(T x);
    T pop();
    T peek();
    boolean isEmpty();
    int size();
    void delete(T n);
}
```

La implementación desarrollada se basó en un arreglo dinámico, encapsulado en la clase `DynamicArray<T>`. Esta decisión permite realizar inserciones, eliminaciones y consultas sobre el tope de la pila de forma eficiente, ya que todas estas operaciones se concentran en el extremo final del arreglo.

Listing 5: Uso de arreglo dinámico en `ArrayStack<T>`

```
private DynamicArray<T> array;

public ArrayStack() {
    array = new DynamicArray<>();
}
```

Las operaciones principales de la pila se implementan directamente sobre la última posición ocupada del arreglo: `push` inserta al final, `pop` elimina el último elemento y `peek` consulta dicho valor sin retirarlo.

Listing 6: Operaciones principales en `MyStack`

```java
public void push(T x) {
    array.add(x);
}

public T pop() {
    return array.removeLast();
}

public T peek() {
    return array.get(array.size() - 1);
}
```

El arreglo subyacente crece dinámicamente cuando se alcanza la capacidad máxima, mediante la creación de un nuevo arreglo de mayor tamaño y la copia de los elementos existentes. Esta estrategia permite mantener una buena eficiencia promedio en las inserciones.

Listing 7: Redimensionamiento del arreglo dinámico

```java
capacity *= 2;
T[] newData = (T[]) new Object[capacity];

for (int i = 0; i < size; i++) {
    newData[i] = data[i];
}
```

Además de las operaciones clásicas de una pila, la interfaz incluye el método `delete`, que elimina la primera aparición de un valor específico. Esta operación no forma parte del comportamiento tradicional de una pila, por lo que su implementación requiere desapilar elementos hasta encontrar el objetivo y luego reconstruir la estructura.

Listing 8: Idea general del método `delete` en `MyStack`

```java
while (!isEmpty()) {
    T value = pop();

    if (value.equals(n)) {
        break;
    }

    temp.add(value);
}
```

En conjunto, esta implementación prioriza la eficiencia de las operaciones fundamentales del modelo LIFO, mientras que operaciones adicionales como `delete` permiten extender el análisis comparativo a escenarios menos restrictivos, a costa de un mayor costo computacional.

3.3. Implementación de `MyQueue<T>`

La estructura `MyQueue<T>` fue definida mediante una interfaz que establece el comportamiento fundamental de una cola, siguiendo el principio FIFO (*First In, First Out*). Al igual que en el caso de la pila, esta interfaz permite desacoplar la definición del comportamiento de la estructura de su implementación concreta.

Listing 9: Contrato definido en `MyQueue<T>`

```
public interface MyQueue<T> {
    void enqueue(T x);
    T dequeue();
    T front();
    boolean isEmpty();
    int size();
    void delete(T n);
}
```

La implementación desarrollada se basó en un arreglo circular dinámico. Esta decisión evita el desplazamiento de elementos al insertar o eliminar, ya que la estructura mantiene dos índices, `front` y `rear`, que representan el inicio y el final lógico de la cola.

Listing 10: Estructura base de la cola circular

```
private T[] data;
private int front;
private int rear;
private int size;
private int capacity;
```

Las operaciones principales se realizan actualizando estos índices mediante aritmética modular. De este modo, `enqueue` inserta un elemento al final lógico de la cola y `dequeue` elimina el elemento ubicado al frente, reutilizando posiciones liberadas dentro del arreglo.

Listing 11: Uso de índices circulares en `MyQueue`

```
data[rear] = x;
rear = (rear + 1) % capacity;

T value = data[front];
front = (front + 1) % capacity;
```

La operación `front` consulta directamente el elemento ubicado al inicio lógico de la estructura, mientras que `isEmpty` y `size` dependen únicamente del estado interno del arreglo y de los contadores mantenidos.

Cuando la capacidad disponible se agota, se realiza un redimensionamiento dinámico. En este proceso, los elementos deben copiarse en un nuevo arreglo conservando su orden lógico a partir de la posición marcada por `front`.

Listing 12: Redimensionamiento preservando el orden lógico

```
for (int i = 0; i < size; i++) {
    newData[i] = data[(front + i) % capacity];
}

data = newData;
front = 0;
rear = size;
```

Al igual que en la pila, se implementó también el método `delete`, el cual permite eliminar la primera aparición de un valor específico. Dado que la cola solo permite acceso directo al frente, esta operación requiere recorrer la estructura mediante sucesivas llamadas a `dequeue` y reinsertar los elementos que no coinciden con el valor buscado.

Listing 13: Idea general del método `delete` en `MyQueue`

```
for (int i = 0; i < currentSize; i++) {
    T value = dequeue();

    if (!deleted && value.equals(n)) {
        deleted = true;
        continue;
    }

    enqueue(value);
}
```

En conjunto, esta implementación prioriza la eficiencia de las operaciones fundamentales del modelo FIFO, aprovechando el comportamiento circular del arreglo para evitar costos innecesarios por desplazamiento. De nuevo, el método `delete` amplía el análisis comparativo del proyecto, aunque introduce un costo mayor que las operaciones principales de la estructura.

4. Métodos implementados

4.1. Métodos de `List`

Los métodos implementados para la estructura `List` fueron definidos a través de la interfaz `ListADT<T>`, la cual establece un contrato común para todas las variantes de listas enlazadas desarrolladas en el proyecto. Esta decisión permite unificar el comportamiento esperado de la estructura, independientemente de si su implementación concreta corresponde a una lista simplemente enlazada, doblemente enlazada, con cola o sin cola.

Listing 14: Métodos declarados en `ListADT<T>`

```java
public interface ListADT<T>{
    void pushFront(T value);
    void pushBack(T value);
    T popFront();
    T popBack();
    boolean isEmpty();
    T topFront();
    T topBack();
    int size();
    Position<T> find(T value);
    void erase(Position<T> position);
    void addBefore(Position<T> position, T value);
    void addAfter(Position<T> position, T value);
}
```

A continuación, se describe la finalidad de cada uno de los métodos definidos en este contrato, así como el motivo de su inclusión dentro de la interfaz.

- `pushFront(T value)`: inserta un nuevo elemento al inicio de la lista. Su firma no retorna ningún valor, ya que su propósito es modificar directamente la estructura agregando un nuevo nodo en la primera posición.
- `pushBack(T value)`: inserta un nuevo elemento al final de la lista. Al igual que `pushFront`, no retorna ningún valor, puesto que su función es actualizar la estructura enlazando el nuevo nodo al extremo final.

- `popFront()`: elimina el primer elemento de la lista y retorna su valor. Se definió con retorno de tipo `T` para que, además de modificar la estructura, permita recuperar el elemento removido.
- `popBack()`: elimina el último elemento de la lista y retorna su valor. Su inclusión permite trabajar de forma simétrica con ambos extremos de la estructura, especialmente útil al comparar implementaciones con y sin referencia al final.
- `isEmpty()`: retorna un valor booleano que indica si la lista se encuentra vacía. Este método es importante como operación de apoyo para validar el estado de la estructura antes de ejecutar inserciones, eliminaciones o consultas.
- `topFront()`: retorna el valor almacenado en el primer nodo sin eliminarlo. Se utiliza como operación de consulta sobre el extremo inicial de la lista.
- `topBack()`: retorna el valor almacenado en el último nodo sin eliminarlo. Al igual que `topFront`, es una operación de consulta, pero orientada al extremo final.
- `size()`: retorna la cantidad de elementos almacenados en la lista. Se definió con retorno entero para consultar el tamaño actual de la estructura sin necesidad de recorrerla.
- `find(T value)`: busca un elemento dentro de la lista y retorna una referencia de tipo `Position<T>` asociada al nodo donde se encuentra dicho valor. Esta decisión de diseño permite separar la búsqueda del acceso directo al nodo, facilitando operaciones posteriores sobre posiciones concretas.
- `erase(Position<T> position)`: elimina de la lista el nodo asociado a una posición previamente obtenida. El uso de `Position<T>` en lugar de un valor directo permite eliminar específicamente un nodo identificado por la estructura, evitando ambigüedades cuando existen elementos repetidos.
- `addBefore(Position<T> position, T value)`: inserta un nuevo elemento antes de una posición dada. Este método permite realizar inserciones relativas a un nodo ya existente, lo cual extiende la flexibilidad de la estructura más allá de los extremos de la lista.
- `addAfter(Position<T> position, T value)`: inserta un nuevo elemento después de una posición dada. Al igual que `addBefore`, su diseño responde a la necesidad de manipular la estructura en posiciones internas y no únicamente en el inicio o en el final.

En conjunto, estos métodos permiten cubrir las operaciones fundamentales de inserción, eliminación, consulta, búsqueda y modificación relativa de posiciones dentro de la lista. La interfaz `ListADT<T>` fue diseñada de esta manera para que todas las implementaciones

compartieran exactamente el mismo comportamiento externo, dejando que las diferencias entre ellas surgieran únicamente en la forma de resolver internamente cada operación y, por tanto, en su complejidad temporal.

4.2. Métodos de `MyStack<T>`

Los métodos implementados para la estructura `MyStack<T>` fueron definidos mediante una interfaz que establece el comportamiento fundamental de una pila. Esta abstracción permite describir las operaciones esenciales asociadas al modelo LIFO (*Last In, First Out*), en el cual el último elemento insertado es el primero en ser removido.

Listing 15: Métodos declarados en `MyStack<T>`

```
public interface MyStack<T> {
    void push(T x);
    T pop();
    T peek();
    boolean isEmpty();
    int size();
    void delete(T n);
}
```

A continuación, se describe la finalidad de cada uno de los métodos definidos en esta interfaz.

- `push(T x)`: inserta un nuevo elemento en el tope de la pila. No retorna ningún valor, ya que su objetivo es modificar directamente la estructura agregando un nuevo elemento en la posición más alta.
- `pop()`: elimina el elemento ubicado en el tope de la pila y retorna su valor. Este método materializa el principio LIFO, puesto que siempre remueve el último elemento insertado.
- `peek()`: retorna el valor del elemento ubicado en el tope de la pila sin eliminarlo. Su inclusión permite consultar el estado inmediato de la estructura sin modificar su contenido.
- `isEmpty()`: retorna un valor booleano que indica si la pila se encuentra vacía. Este método es útil para validar el estado de la estructura antes de ejecutar operaciones como `pop()` o `peek()`.
- `size()`: retorna la cantidad de elementos almacenados en la pila. Permite conocer el tamaño actual de la estructura sin necesidad de recorrerla.

- `delete(T n)`: elimina la primera aparición del valor indicado dentro de la pila. Aunque esta no es una operación clásica en una pila abstracta, fue incluida en el contrato del proyecto para permitir comparaciones adicionales entre estructuras y analizar el costo de eliminar un elemento específico dentro de una organización LIFO.

En conjunto, estos métodos cubren tanto las operaciones fundamentales de una pila como una operación adicional de eliminación por valor. La interfaz `MyStack<T>` fue diseñada de esta manera para proporcionar un contrato uniforme sobre el cual pudiera construirse una implementación concreta y, posteriormente, analizar su comportamiento teórico y empírico.

4.3. Métodos de `MyQueue<T>`

Los métodos implementados para la estructura `MyQueue<T>` fueron definidos mediante una interfaz que establece el comportamiento fundamental de una cola. Esta abstracción responde al modelo FIFO (*First In, First Out*), en el cual el primer elemento en ingresar a la estructura es también el primero en ser removido.

Listing 16: Métodos declarados en `MyQueue<T>`

```
public interface MyQueue<T> {
    void enqueue(T x);
    T dequeue();
    T front();
    boolean isEmpty();
    int size();
    void delete(T n);
}
```

A continuación, se describe la finalidad de cada uno de los métodos definidos en esta interfaz.

- `enqueue(T x)`: inserta un nuevo elemento al final lógico de la cola. No retorna ningún valor, ya que su función es modificar la estructura agregando un nuevo elemento en la posición de entrada.
- `dequeue()`: elimina el elemento ubicado al frente de la cola y retorna su valor. Este método refleja el comportamiento FIFO, puesto que siempre remueve el elemento que lleva más tiempo dentro de la estructura.
- `front()`: retorna el valor del elemento ubicado al frente de la cola sin eliminarlo. Se trata de una operación de consulta que permite inspeccionar el siguiente elemento a ser removido.

- `isEmpty()`: retorna un valor booleano que indica si la cola se encuentra vacía. Este método resulta útil para validar el estado de la estructura antes de ejecutar operaciones como `dequeue()` o `front()`.
- `size()`: retorna la cantidad de elementos almacenados en la cola. Permite conocer el tamaño actual de la estructura sin necesidad de recorrer sus elementos.
- `delete(T n)`: elimina la primera aparición del valor indicado dentro de la cola. Al igual que en la pila, esta no corresponde a una operación elemental del modelo abstracto clásico, pero fue incorporada en el contrato del proyecto con el propósito de extender el análisis de comportamiento y comparar el costo de eliminar un valor específico en una estructura FIFO.

En conjunto, estos métodos cubren las operaciones esenciales de una cola y una operación adicional de eliminación por valor. La interfaz `MyQueue<T>` define así un contrato uniforme que permite implementar la estructura de manera concreta y, posteriormente, estudiar su comportamiento desde un punto de vista teórico y experimental.

5. Análisis de complejidad

5.1. Análisis teórico

5.1.1. Complejidad teórica de `ListADT<T>`

El análisis de complejidad teórica de la estructura `ListADT<T>` se realizó considerando las cuatro variantes implementadas: lista simplemente enlazada sin cola (`Singly`), lista simplemente enlazada con cola (`SinglyTail`), lista doblemente enlazada sin cola (`Doubly`) y lista doblemente enlazada con cola (`DoublyTail`).

Dado que todas las implementaciones comparten el mismo contrato, las diferencias en complejidad provienen exclusivamente de su representación interna, en particular del uso de la referencia `tail` y de la presencia de enlaces dobles mediante `prev`.

En la Tabla 1 se presenta la complejidad temporal esperada para cada uno de los métodos definidos en la interfaz, suponiendo que se dispone directamente de la posición requerida en los métodos que operan sobre `Position<T>`.

Cuadro 1: Complejidad teórica de los métodos de `ListADT<T>` según implementación

Método	**Singly**	**SinglyTail**	**Doubly**	**DoublyTail**
`pushFront`	$O(1)$	$O(1)$	$O(1)$	$O(1)$
`pushBack`	$O(n)$	$O(1)$	$O(n)$	$O(1)$
`popFront`	$O(1)$	$O(1)$	$O(1)$	$O(1)$
`popBack`	$O(n)$	$O(n)$	$O(n)$	$O(1)$
`topFront`	$O(1)$	$O(1)$	$O(1)$	$O(1)$
`topBack`	$O(n)$	$O(1)$	$O(n)$	$O(1)$
`isEmpty`	$O(1)$	$O(1)$	$O(1)$	$O(1)$
`size`	$O(1)$	$O(1)$	$O(1)$	$O(1)$
`find`	$O(n)$	$O(n)$	$O(n)$	$O(n)$
`erase`*	$O(n)$	$O(n)$	$O(1)$	$O(1)$
`addBefore`*	$O(n)$	$O(n)$	$O(1)$	$O(1)$
`addAfter`*	$O(1)$	$O(1)$	$O(1)$	$O(1)$

**Las complejidades indicadas corresponden al costo de la operación suponiendo que se dispone previamente de una referencia válida de tipo `Position<T>`.*

A partir de la tabla anterior, se observa que las operaciones sobre el inicio de la lista, como `pushFront`, `popFront` y `topFront`, presentan complejidad constante en todas las implementaciones, debido a que siempre existe acceso directo al primer nodo mediante la referencia `head`.

En contraste, las operaciones sobre el extremo final dependen de la existencia de una referencia `tail`. En las implementaciones sin cola (`Singly` y `Doubly`), operaciones como `pushBack` y `topBack` requieren recorrer la lista desde `head` hasta el último nodo, lo que implica un costo lineal. En las implementaciones con cola, este recorrido se evita, permitiendo resolver dichas operaciones en tiempo constante.

El caso de `popBack` evidencia una diferencia adicional entre las estructuras. En `SinglyTail`, a pesar de contar con una referencia `tail`, no es posible eliminar el último nodo en tiempo constante, ya que no se dispone de acceso directo al nodo anterior. Por ello, es necesario recorrer la lista hasta el penúltimo nodo. En cambio, en `DoublyTail`, la presencia de la referencia `prev` permite acceder directamente al nodo anterior a `tail`, logrando una complejidad constante.

Por otro lado, el método `find` presenta complejidad lineal en todas las implementaciones, dado que la búsqueda se realiza de manera secuencial desde `head` hasta encontrar el elemento o alcanzar el final de la lista.

Los métodos `erase`, `addBefore` y `addAfter` requieren un análisis particular. Cuando se dispone de una referencia directa al nodo objetivo, `addAfter` puede resolverse en tiempo constante en todas las implementaciones, ya que solo implica modificar enlaces locales. En el caso de `erase` y `addBefore`, las listas doblemente enlazadas permiten realizar estas operaciones en tiempo constante gracias a la referencia `prev`, mientras que en las listas simplemente enlazadas es necesario ubicar el nodo anterior mediante un recorrido adicional.

No obstante, en el uso práctico de la estructura, es frecuente que la posición sobre la cual se desea operar no esté disponible previamente, por lo que debe obtenerse mediante una llamada a `find`. Esto introduce un *overhead* adicional que afecta el costo total de la operación.

Con el fin de reflejar este escenario más cercano al uso real, en la Tabla 2 se presenta la complejidad esperada al considerar el flujo completo que incluye la búsqueda previa.

Cuadro 2: Complejidad total considerando búsqueda previa con `find`

Operación compuesta	**Singly**	**SinglyTail**	**Doubly**	**DoublyTail**
`find + erase`	$O(n)$	$O(n)$	$O(n)$	$O(n)$
`find + addBefore`	$O(n)$	$O(n)$	$O(n)$	$O(n)$
`find + addAfter`	$O(n)$	$O(n)$	$O(n)$	$O(n)$

De acuerdo con lo anterior, aunque ciertas operaciones pueden ejecutarse en tiempo constante cuando se dispone de la posición objetivo, el costo total en escenarios reales suele estar dominado por el tiempo de búsqueda. Este fenómeno resulta especialmente relevante en el análisis empírico, donde la medición del tiempo de ejecución debe considerar este *overhead* para reflejar de manera fiel el comportamiento de la estructura en situaciones prácticas.

5.1.2. Complejidad teórica de `MyStack<T>`

La estructura `MyStack<T>` fue implementada mediante un arreglo dinámico, lo que permite realizar operaciones sobre el tope de la pila de forma eficiente. Dado que la pila sigue el principio LIFO (*Last In, First Out*), todas las operaciones principales se concentran en el extremo final del arreglo.

En la Tabla 3 se presenta la complejidad temporal esperada de los métodos definidos en la interfaz.

Cuadro 3: Complejidad teórica de los métodos de `MyStack<T>`

Método	**Complejidad**
`push`	$O(1)$ amortizado
`pop`	$O(1)$
`peek`	$O(1)$
`isEmpty`	$O(1)$
`size`	$O(1)$
`delete`	$O(n)$

Las operaciones `push`, `pop` y `peek` se realizan directamente sobre el último elemento del arreglo dinámico, por lo que su costo es constante. Sin embargo, la operación `push` se clasifica como $O(1)$ amortizado, ya que en situaciones donde el arreglo alcanza su capacidad máxima, es necesario redimensionarlo, lo que implica copiar todos los elementos a un nuevo arreglo de mayor tamaño.

Las operaciones `isEmpty` y `size` también se resuelven en tiempo constante, ya que dependen únicamente de variables internas que mantienen el estado de la estructura.

Por otro lado, el método `delete` introduce un comportamiento distinto al de una pila clásica. Para eliminar un elemento específico, es necesario desapilar elementos hasta encontrar el valor buscado y luego reconstruir la pila con los elementos restantes. Este proceso implica recorrer una cantidad proporcional de elementos, por lo que su complejidad en el peor caso es $O(n)$.

En consecuencia, la implementación de `MyStack<T>` ofrece un desempeño eficiente en sus operaciones fundamentales, mientras que operaciones adicionales como `delete` presentan un costo mayor debido a las restricciones inherentes al modelo LIFO.

5.1.3. Complejidad teórica de `MyQueue<T>`

La estructura `MyQueue<T>` fue implementada mediante un arreglo circular dinámico, lo que permite optimizar el uso del espacio y evitar desplazamientos de elementos durante las operaciones. Esta estructura sigue el principio FIFO (*First In, First Out*), en el cual el primer elemento en ingresar es el primero en salir.

En la Tabla 4 se presenta la complejidad temporal esperada de los métodos definidos en la interfaz.

Cuadro 4: Complejidad teórica de los métodos de `MyQueue<T>`

Método	**Complejidad**
`enqueue`	$O(1)$ amortizado
`dequeue`	$O(1)$
`front`	$O(1)$
`isEmpty`	$O(1)$
`size`	$O(1)$
`delete`	$O(n)$

Las operaciones principales de la cola, `enqueue` y `dequeue`, se realizan mediante el uso de índices circulares (`rear` y `front`), lo que permite insertar y eliminar elementos sin necesidad de desplazar el contenido del arreglo. Esto garantiza un costo constante para estas operaciones en condiciones normales.

Al igual que en el caso de la pila, la operación `enqueue` es de tipo amortizado debido al proceso de redimensionamiento del arreglo cuando se alcanza la capacidad máxima. Durante este proceso, los elementos deben copiarse a un nuevo arreglo manteniendo su orden lógico, lo que introduce un costo puntual de $O(n)$.

Las operaciones `front`, `isEmpty` y `size` se ejecutan en tiempo constante, ya que dependen únicamente de accesos directos a posiciones del arreglo o a variables internas.

El método `delete`, al igual que en la pila, no corresponde a una operación clásica del modelo FIFO. Su implementación requiere recorrer la cola completa mediante sucesivas operaciones de `dequeue` y `enqueue`, lo que implica un número de operaciones proporcional al tamaño de la estructura. Por esta razón, su complejidad es $O(n)$.

En términos generales, la implementación de `MyQueue<T>` mediante arreglo circular ofrece un desempeño eficiente en sus operaciones fundamentales, evitando costos adicionales asociados al desplazamiento de elementos, mientras que las operaciones no estándar como `delete` presentan un costo lineal.

5.2. Metodología experimental

Con el propósito de contrastar el análisis teórico de complejidad con el comportamiento observado en la práctica, se diseñó una metodología experimental basada en la medición del tiempo de ejecución de las operaciones implementadas sobre listas, pilas y colas.

5.2.1. Estrategia general de medición

La medición experimental se implementó en Java mediante un conjunto de clases específicas de benchmark, organizadas por tipo de estructura: `ListBenchmark`, `StackBenchmark` y `QueueBenchmark`. Estas clases delegan la ejecución repetida de las pruebas a la clase `BenchmarkRunner`, la cual se encarga de realizar corridas de calentamiento (*warmup*), repeticiones medidas y cálculo de estadísticas descriptivas.

Cada medición individual se realizó utilizando `System.nanoTime()`, encapsulado en la utilidad `Timer.measure(...)`, con el fin de registrar el tiempo total de ejecución de una tarea determinada en nanosegundos.

Listing 17: Medición de tiempo usando alta precisión

```
public static long measure(Runnable task) {
    long start = System.nanoTime();
    task.run();
    long end = System.nanoTime();
    return end - start;
}
```

El uso de nanosegundos se justifica debido a la necesidad de alta precisión en la medición de operaciones de corta duración. Dado que muchas de las operaciones evaluadas tienen costos muy bajos (especialmente aquellas de orden constante), el uso de unidades de menor resolución, como milisegundos, podría ocultar diferencias relevantes entre implementaciones.

5.2.2. Ejecución de múltiples repeticiones

Para cada tamaño de entrada y cada operación, la medición no se basa en una única ejecución, sino en múltiples repeticiones independientes. En particular, se realizaron 2 corridas de calentamiento y 5 corridas medidas.

La clase `BenchmarkRunner` centraliza este proceso, ejecutando repetidamente la tarea a medir y calculando estadísticas como promedio, mediana, mínimo y máximo.

Listing 18: Cálculo del promedio a partir de múltiples ejecuciones

```
for (int i = 0; i < measuredRuns; i++) {
    long value = measuredRun.getAsLong();
    samples[i] = value;
    sum += value;
}

long average = sum / measuredRuns;
```

El valor reportado en los archivos CSV y en las gráficas corresponde al promedio de las 5 mediciones realizadas. Este enfoque permite reducir la variabilidad inherente a una única ejecución y obtener resultados más representativos del comportamiento real de cada operación.

5.2.3. Configuración de las corridas

La configuración de las pruebas se controla mediante propiedades del sistema, lo que permite ajustar fácilmente el número de repeticiones sin modificar el código fuente.

Listing 19: Configuración dinámica de repeticiones

```
public static int warmupRuns() {
    return Integer.parseInt(System.getProperty("benchmark.warmup"
        , "1"));
}

public static int measuredRuns() {
    return Integer.parseInt(System.getProperty("benchmark.
        repetitions", "2"));
}
```

Para este trabajo se configuraron:

- 2 corridas de calentamiento (*warmup*)
- 5 repeticiones medidas por cada tamaño de entrada

5.2.4. Conjunto de tamaños de entrada

Con el fin de balancear la representatividad de los resultados con la viabilidad computacional de las pruebas, se definieron tres conjuntos de tamaños de entrada, utilizados según el costo esperado de cada operación:

- **Base:** $10,\ 10^2,\ 10^3,\ 10^4,\ 10^5,\ 10^6$
- **Expensive:** $10,\ 10^2,\ 10^3,\ 10^4,\ 5 \times 10^4,\ 10^5$
- **Very Expensive:** $10,\ 10^2,\ 10^3,\ 10^4,\ 5 \times 10^4$

Esta diferenciación se adoptó debido a que, en pruebas preliminares, algunas operaciones de complejidad lineal o superior comenzaban a presentar tiempos de ejecución excesivos a partir de tamaños del orden de 10^5, lo que hacía inviable extender la medición a tamaños mayores sin incurrir en tiempos desproporcionados.

5.2.5. Criterio de construcción de las pruebas

Para cada estructura y operación, se definió una rutina de medición específica. En general, las pruebas se realizaron sobre estructuras recién creadas en cada corrida, con el fin de evitar efectos de estado residual entre ejecuciones.

Dependiendo de la operación, la medición corresponde al tiempo total de una secuencia de acciones. Por ejemplo:

- en inserciones, se ejecuta repetidamente la operación sobre una estructura inicialmente vacía;
- en eliminaciones o consultas, primero se construye una estructura de tamaño n y luego se ejecuta la operación;
- en operaciones compuestas, se mide una secuencia que incluye búsqueda y modificación.

En consecuencia, los tiempos reportados representan el costo de bloques de ejecución y no el tiempo aislado de una única operación elemental.

5.2.6. Consideraciones sobre operaciones compuestas

En el caso de las listas, operaciones como `erase`, `addBefore` y `addAfter` dependen de una posición de tipo `Position<T>`. En el contexto experimental, dicha posición se obtuvo mediante llamadas previas a `find`, por lo que el tiempo medido incluye también el costo de búsqueda.

De forma similar, en `MyStack<T>` y `MyQueue<T>`, el método `delete` se implementa como una operación compuesta que recorre la estructura completa, lo que explica su comportamiento empírico de orden lineal.

5.2.7. Registro de resultados

Los resultados de cada benchmark se almacenaron en archivos CSV independientes por operación y estructura, con el formato:

```
size,avg_time_ns,median_ns,min_ns,max_ns
```

Esto permite separar claramente la fase de medición de la fase de análisis y facilita la generación de visualizaciones sin necesidad de repetir los experimentos.

5.2.8. Generación de gráficas

La visualización de resultados se realizó mediante scripts en Python utilizando `pandas` y `matplotlib`. Esta etapa se ejecutó de forma independiente al proceso de medición, con el

fin de evitar la introducción de *overhead* en los tiempos registrados.

Se utilizaron escalas logarítmicas en el eje de entrada para facilitar la identificación de tendencias de crecimiento y la comparación entre implementaciones.

5.2.9. Limitaciones metodológicas

A pesar de que la metodología permite identificar patrones consistentes entre el comportamiento teórico y empírico de las estructuras, los resultados deben interpretarse como estimaciones descriptivas. Adicionalmente, varias mediciones corresponden a secuencias de operaciones y no a operaciones unitarias aisladas.

Finalmente, no todas las operaciones se evaluaron con el mismo tamaño máximo de entrada, debido a restricciones de viabilidad computacional. No obstante, esta decisión no afecta la identificación de tendencias de crecimiento ni la comparación relativa entre implementaciones.

5.3. Análisis empírico

En esta sección se presentan los resultados obtenidos a partir de los benchmarks realizados sobre las estructuras `List`, `MyStack<T>` y `MyQueue<T>`. El objetivo es analizar el comportamiento observado y contrastarlo con el análisis teórico de complejidad.

Los valores presentados en las gráficas y tablas corresponden a los resultados obtenidos mediante el framework de benchmark descrito en la metodología experimental. En particular, para cada operación se reporta el tiempo promedio registrado para cada tamaño de entrada, el cual se utilizó como referencia principal para el análisis.

Es importante resaltar que las gráficas muestran el comportamiento empírico de rutinas de ejecución sobre estructuras de tamaño creciente. Por esta razón, el crecimiento observado en los tiempos no siempre representa el costo de una única operación elemental aislada, sino el costo total asociado a secuencias controladas de operaciones, según la forma en que fue definido cada benchmark.

A partir de esta base, el análisis empírico se organizó en varias partes. En primer lugar, se estudia el comportamiento experimental de `MyStack<T>` y `MyQueue<T>` de manera individual. Posteriormente, se comparan ambas estructuras entre sí a través de operaciones equivalentes. Finalmente, se analiza el comportamiento de las distintas implementaciones de `List`, tanto mediante comparaciones directas por método como a través de una visualización global complementaria.

5.3.1. Análisis empírico de `MyStack<T>`

En esta subsección se analizan los resultados experimentales obtenidos para las operaciones implementadas en `MyStack<T>`. Como la pila fue construida sobre un arreglo dinámico, desde el análisis teórico se espera que sus operaciones fundamentales tengan un desempeño eficiente, especialmente aquellas que se realizan directamente sobre el tope de la estructura.

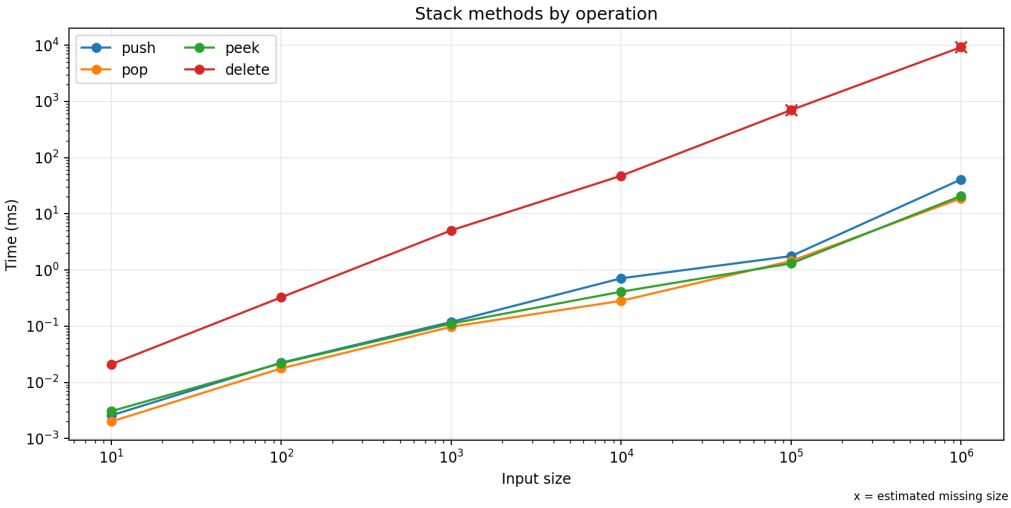

Figura 1: Comportamiento experimental de las operaciones de `MyStack<T>`.

La Figura 1 permite ver con claridad dos comportamientos distintos. Por un lado, las operaciones `push`, `pop` y `peek` siguen trayectorias bastante cercanas entre sí. En los tres casos, el tiempo aumenta de forma gradual conforme crece el tamaño de entrada, pero sin mostrar cambios bruscos ni separaciones importantes entre curvas. Esto resulta coherente con la implementación, ya que las tres operaciones trabajan sobre el extremo final del arreglo dinámico y, en consecuencia, mantienen un costo constante amortizado por operación.

Conviene aclarar, sin embargo, que la gráfica no representa el costo de una única llamada aislada a cada método. Lo que se está midiendo aquí es el tiempo total de ejecutar múltiples operaciones sobre estructuras de tamaño creciente. Por esa razón, el crecimiento observado no contradice el análisis teórico: aunque cada operación individual sea eficiente, el costo acumulado naturalmente aumenta al repetirla muchas veces.

Dentro de este primer grupo, `push` merece una observación particular. Aun cuando esta operación depende del redimensionamiento del arreglo cuando la capacidad se agota, en la práctica su curva se mantiene estable y muy próxima a las de `pop` y `peek`. Esto sugiere que el costo adicional de redimensionar, aunque existe, no domina el comportamiento general en la escala analizada. En otras palabras, el efecto amortizado esperado teóricamente también se refleja de manera razonable en los resultados experimentales.

El caso de `delete` es diferente y destaca inmediatamente en la gráfica. Su tiempo de ejecución es considerablemente mayor que el de las demás operaciones, y además su crecimiento es mucho más pronunciado a medida que aumenta el tamaño de la pila. Esta diferencia tiene una explicación directa en la forma en que fue implementado el método. A diferencia de `push`, `pop` o `peek`, `delete` no corresponde a una operación elemental del modelo LIFO. Para eliminar un valor específico, la pila debe desapilar elementos hasta encontrarlo y, después, reconstruir parcialmente la estructura con los elementos retirados. En consecuencia, el costo total de la operación depende linealmente del número de elementos involucrados, lo que justifica que su curva se aleje con tanta claridad del resto.

En conjunto, los resultados obtenidos para `MyStack<T>` muestran una correspondencia clara entre teoría y práctica. Las operaciones fundamentales de la pila conservan un comportamiento eficiente y estable, mientras que la operación adicional `delete` introduce un costo significativamente mayor debido a que exige recorrer la estructura. Así, la evidencia experimental no solo confirma la complejidad esperada, sino que también permite visualizar de forma concreta la diferencia entre las operaciones propias del modelo de pila y aquellas que se apartan de él.

5.3.2. Análisis empírico de `MyQueue<T>`

En esta subsección se analizan los resultados experimentales obtenidos para las operaciones implementadas en `MyQueue<T>`. En este caso, la cola fue implementada mediante un arreglo circular dinámico, lo que permite realizar inserciones y eliminaciones de forma eficiente en extremos opuestos de la estructura.

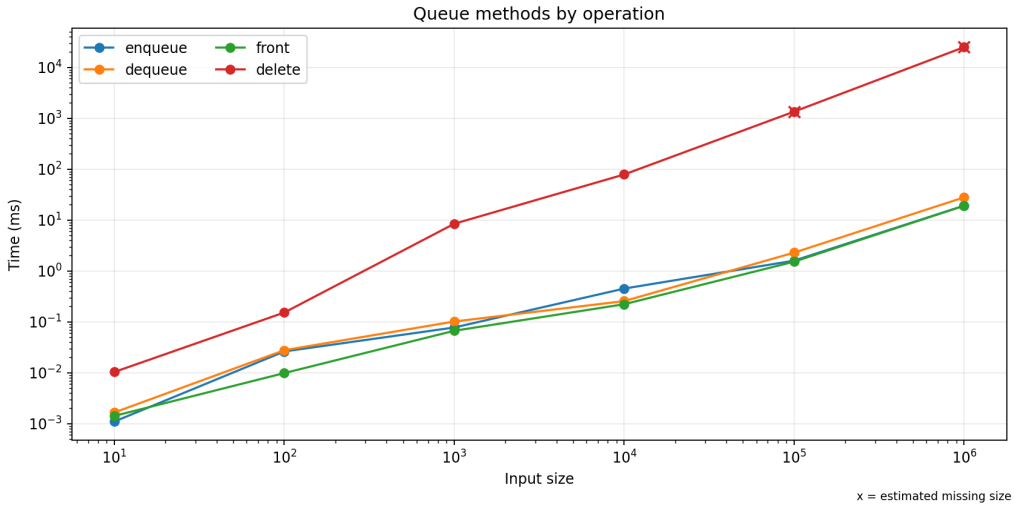

Figura 2: Comportamiento experimental de las operaciones de `MyQueue<T>`.

A partir de la Figura 2 se distinguen, nuevamente, dos comportamientos bien definidos. Por un lado, las operaciones `enqueue`, `dequeue` y `front` presentan curvas muy cercanas

entre sí, con un crecimiento progresivo y estable conforme aumenta el tamaño de entrada. Este patrón es consistente con la implementación basada en un arreglo circular, en la cual tanto la inserción como la eliminación se realizan en tiempo constante amortizado, y el acceso al frente no requiere recorridos adicionales.

Al igual que en el caso de la pila, es importante recordar que los tiempos mostrados en la gráfica corresponden al costo total de ejecutar múltiples operaciones sobre estructuras de tamaño creciente. En consecuencia, el crecimiento observado no refleja el costo individual de una operación aislada, sino el efecto acumulado de repetirla varias veces bajo condiciones controladas.

Dentro de este grupo, se observa que `enqueue` y `dequeue` mantienen un comportamiento prácticamente paralelo, lo que indica que ambas operaciones tienen costos similares en la práctica. Por su parte, `front` se mantiene ligeramente por debajo en algunos puntos, lo cual resulta razonable dado que se trata de una operación de acceso que no modifica la estructura.

La operación `delete`, en contraste, presenta un comportamiento claramente diferenciado. Su curva se ubica muy por encima de las demás y muestra un crecimiento mucho más pronunciado a medida que aumenta el tamaño de la cola. Este resultado se explica por la naturaleza de su implementación: al no ser una operación elemental del modelo FIFO, `delete` requiere recorrer la estructura para localizar el elemento objetivo y reconstruir la cola, lo que introduce un costo lineal respecto al número de elementos.

En conjunto, los resultados experimentales de `MyQueue<T>` confirman las expectativas teóricas. Las operaciones fundamentales de la cola (`enqueue`, `dequeue` y `front`) mantienen un comportamiento eficiente y estable, mientras que la operación `delete` presenta un costo significativamente mayor debido a que implica un recorrido completo de la estructura.

5.3.3. Comparación entre `MyStack<T>` y `MyQueue<T>`

En esta subsección se comparan experimentalmente las operaciones equivalentes entre `MyStack<T>` y `MyQueue<T>`. Aunque ambas estructuras responden a modelos distintos (LIFO y FIFO, respectivamente), sus implementaciones basadas en arreglos dinámicos permiten analizar su desempeño bajo un mismo contexto.

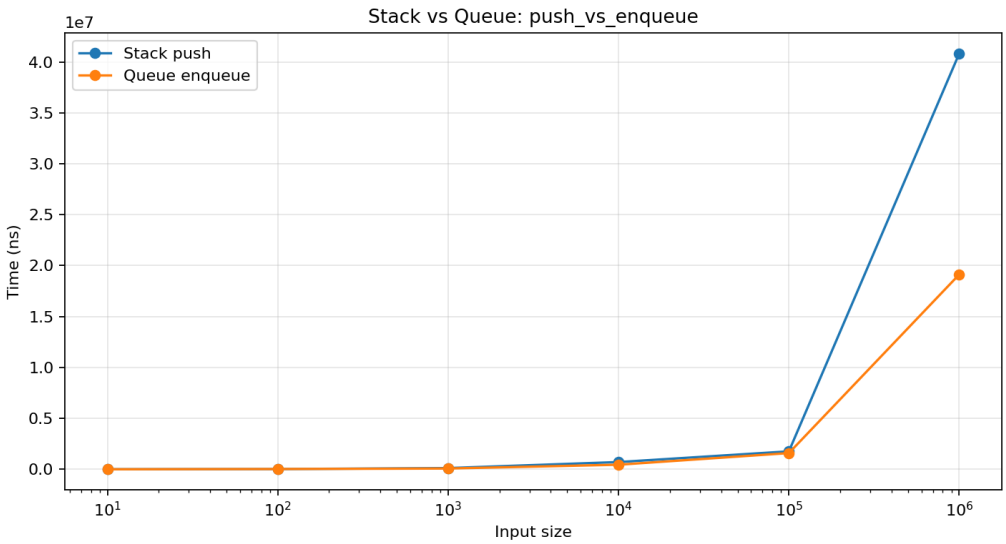

Figura 3: Comparación entre `push` y `enqueue`.

Inserción: `push` vs `enqueue` En la Figura 3 se observa que ambas operaciones presentan un comportamiento muy similar a lo largo de todos los tamaños de entrada. Esto es consistente con su implementación en arreglos dinámicos, donde ambas operaciones corresponden a inserciones en extremos de la estructura y tienen un costo amortizado constante.

Para tamaños pequeños y medianos, las diferencias entre ambas curvas son prácticamente despreciables. Sin embargo, para tamaños más grandes se aprecia que `enqueue` tiende a presentar tiempos ligeramente menores que `push`. Esta diferencia puede atribuirse a detalles de implementación, como el manejo del índice circular en la cola, aunque no altera la tendencia general.

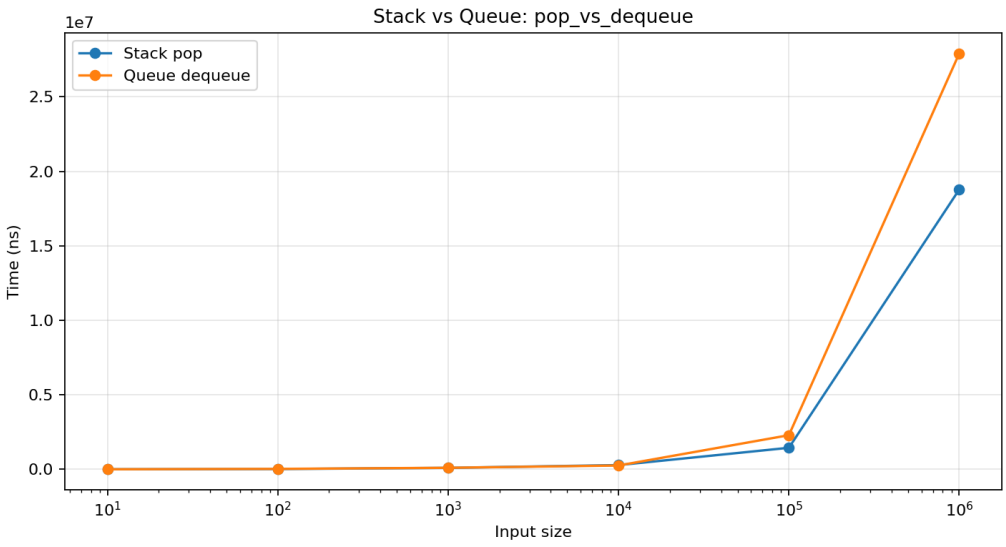

Figura 4: Comparación entre `pop` y `dequeue`.

Eliminación: `pop` vs `dequeue` La Figura 4 muestra que ambas operaciones presentan un crecimiento similar conforme aumenta el tamaño de entrada, lo cual es coherente con su complejidad amortizada constante. No obstante, se observa que `dequeue` presenta tiempos ligeramente superiores a `pop` en tamaños grandes.

Este comportamiento puede explicarse por el hecho de que `dequeue` requiere gestionar el índice circular del arreglo, mientras que `pop` opera directamente sobre el final del arreglo. Aun así, ambas operaciones mantienen un comportamiento eficiente en términos generales.

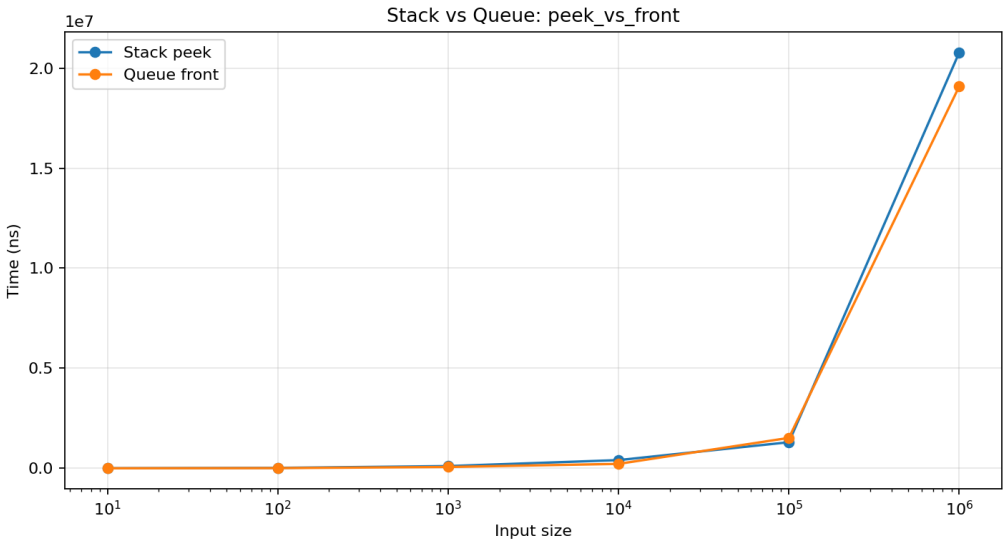

Figura 5: Comparación entre `peek` y `front`.

Consulta: `peek` vs `front` En la Figura 5 se evidencia que las operaciones `peek` y `front` presentan un comportamiento prácticamente idéntico. Ambas corresponden a accesos directos al elemento relevante (tope o frente), sin modificar la estructura, por lo que su costo es constante.

Las pequeñas diferencias observadas en algunos tamaños pueden atribuirse a variaciones del entorno de ejecución o detalles menores de implementación, pero no representan diferencias significativas en términos de complejidad.

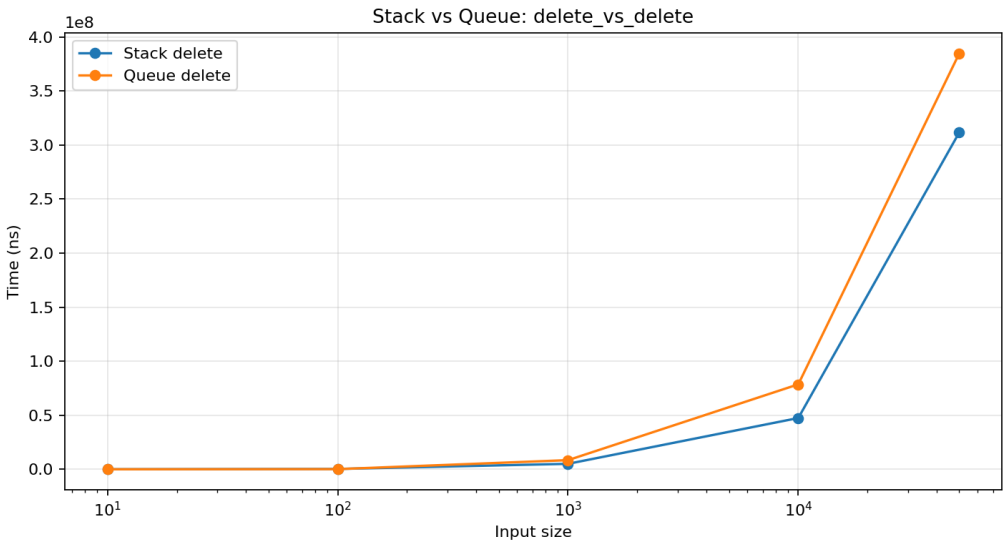

Figura 6: Comparación entre `delete` en Stack y Queue.

Operación adicional: `delete` Finalmente, la Figura 6 muestra un comportamiento claramente distinto al de las operaciones anteriores. En ambos casos, la operación `delete` presenta un crecimiento mucho más pronunciado, lo que indica un costo significativamente mayor.

Esto se debe a que `delete` no es una operación propia de los modelos LIFO o FIFO, por lo que requiere recorrer la estructura para localizar el elemento y reorganizar los datos. En este contexto, se observa que `delete` en la cola presenta un costo aún mayor que en la pila, lo cual puede explicarse por el proceso adicional de reencolado de elementos.

Síntesis En conjunto, los resultados experimentales muestran que las operaciones fundamentales de `MyStack<T>` y `MyQueue<T>` presentan desempeños comparables, con comportamientos consistentes con su análisis teórico. Las diferencias más notorias aparecen en operaciones no estándar como `delete`, donde ambas estructuras deben abandonar su patrón de acceso principal y realizar recorridos adicionales, lo que impacta significativamente su rendimiento.

5.3.4. Análisis empírico de `List`

En esta subsección se analizan los resultados experimentales obtenidos para las distintas implementaciones de la interfaz `ListADT`. Se consideran cuatro variantes: lista simplemente enlazada sin cola, simplemente enlazada con cola, doblemente enlazada sin cola y doblemente enlazada con cola.

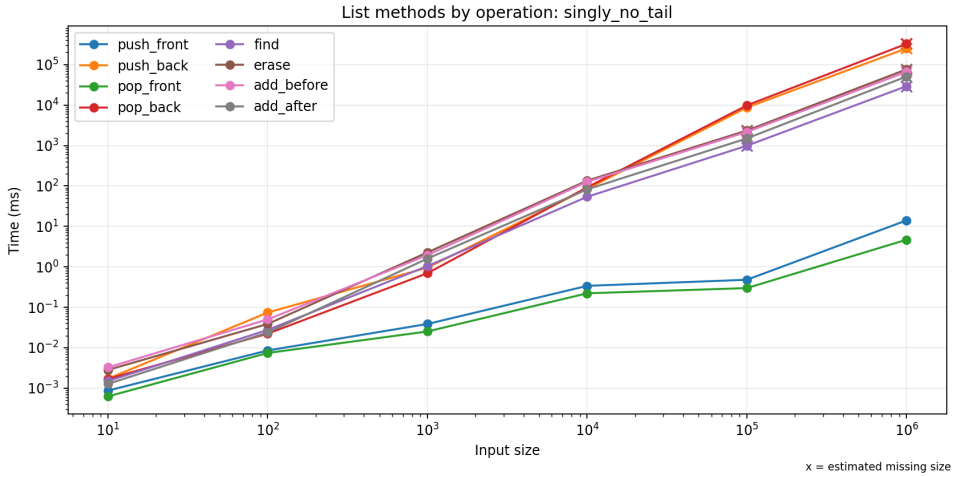

Figura 7: Comportamiento de las operaciones en lista simplemente enlazada sin cola.

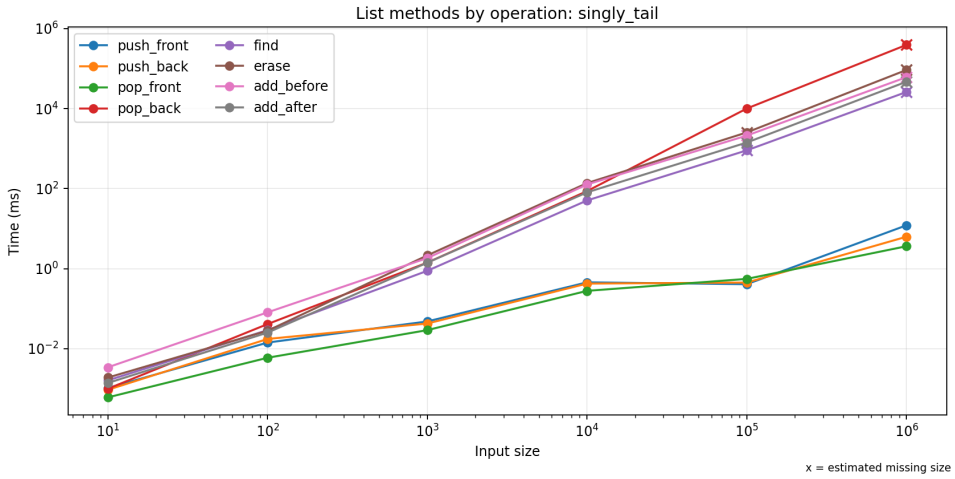

Figura 8: Comportamiento de las operaciones en lista simplemente enlazada con cola.

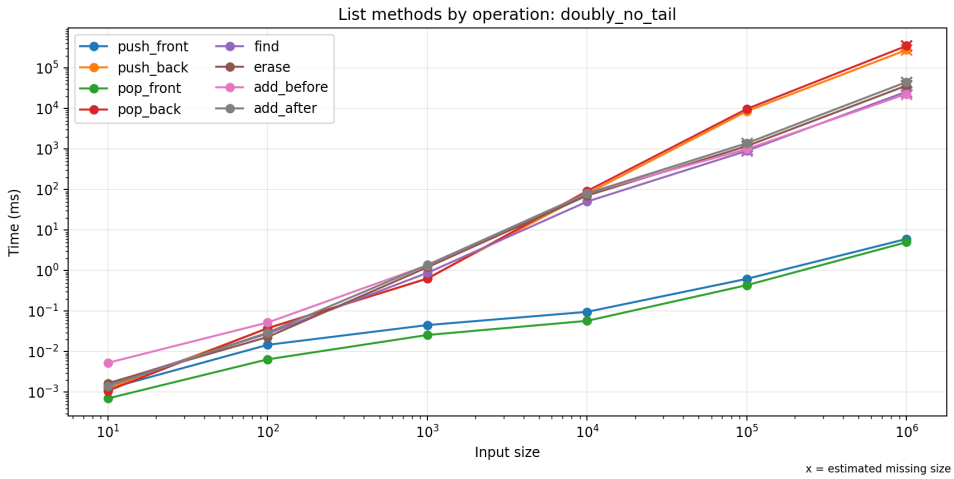

Figura 9: Comportamiento de las operaciones en lista doblemente enlazada sin cola.

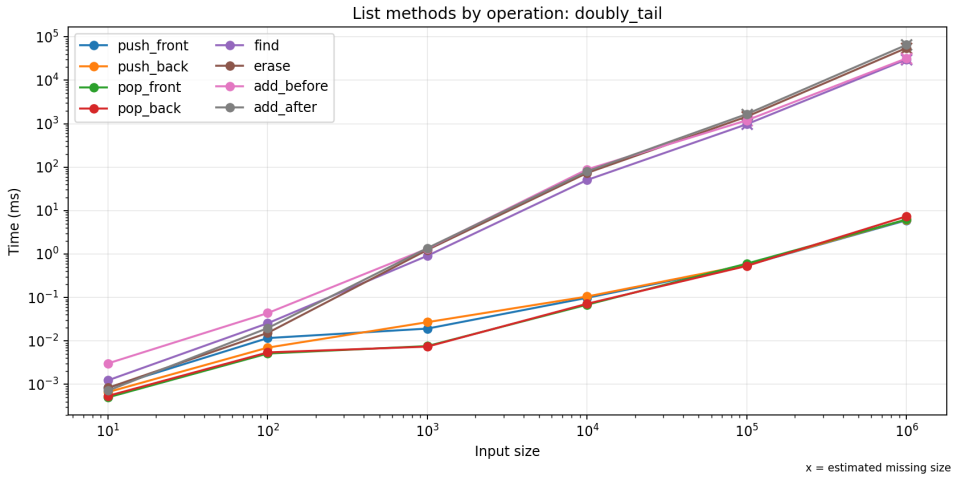

Figura 10: Comportamiento de las operaciones en lista doblemente enlazada con cola.

A partir de las Figuras 7–10 se observa un patrón consistente en todas las implementaciones: las operaciones pueden agruparse en dos grandes categorías según su comportamiento temporal.

Por un lado, operaciones como `push_front` y `pop_front` presentan un crecimiento suave y controlado a medida que aumenta el tamaño de entrada. Este comportamiento es coherente con un costo constante por operación, ya que únicamente implican modificaciones en el nodo cabeza sin necesidad de recorrer la estructura.

Por otro lado, operaciones como `find`, `erase`, `add_before` y `add_after` muestran un crecimiento mucho más pronunciado. Esto se debe a que todas ellas dependen de la localización de un nodo dentro de la lista, lo que implica recorrer la estructura. En consecuencia, su costo está dominado por un comportamiento lineal en función del tamaño de la lista.

Un caso particularmente relevante es el de las operaciones relacionadas con el final de la

lista. En las implementaciones que no mantienen una referencia al nodo final, `push_back` y especialmente `pop_back` presentan un crecimiento significativo. Esto ocurre porque es necesario recorrer la lista completa para acceder al último elemento o a su predecesor.

En contraste, cuando la estructura incluye una referencia a la cola (*tail*), se observa una mejora considerable en el desempeño de estas operaciones. En particular, `push_back` pasa a comportarse como una operación de costo constante, mientras que `pop_back` mejora notablemente en las listas doblemente enlazadas con cola, gracias al uso de referencias hacia el nodo anterior (`prev`).

Al comparar las implementaciones simplemente y doblemente enlazadas, se evidencia que las listas doblemente enlazadas ofrecen ventajas en operaciones que requieren modificar enlaces en ambas direcciones, como `erase` o `add_before`. La disponibilidad de referencias al nodo previo permite realizar estas operaciones de manera más directa, evitando recorridos adicionales o simplificando la actualización de enlaces.

En conjunto, los resultados experimentales confirman el análisis teórico de las listas enlazadas. Las operaciones que acceden directamente a extremos conocidos de la estructura presentan un comportamiento eficiente, mientras que aquellas que requieren búsqueda o acceso posicional introducen un costo lineal. Asimismo, se evidencia que decisiones de diseño como mantener una referencia a cola o utilizar una estructura doblemente enlazada tienen un impacto significativo en el desempeño de operaciones específicas.

5.3.5. Comparativa entre implementaciones de `List`

Además del análisis individual de cada implementación, resulta útil comparar directamente el efecto de ciertas decisiones de diseño sobre el desempeño de las operaciones. En particular, una de las diferencias más importantes entre las variantes desarrolladas es la presencia o ausencia de una referencia al último nodo de la lista (`tail`).

Con el fin de aislar este efecto, en esta subsección se comparan las implementaciones simplemente enlazadas con y sin cola, y las implementaciones doblemente enlazadas con y sin cola. Se seleccionaron principalmente las operaciones `push_back`, `pop_back` y `find`, ya que permiten observar tanto los casos en los que `tail` introduce una mejora significativa como aquellos en los que su presencia no modifica sustancialmente el comportamiento.

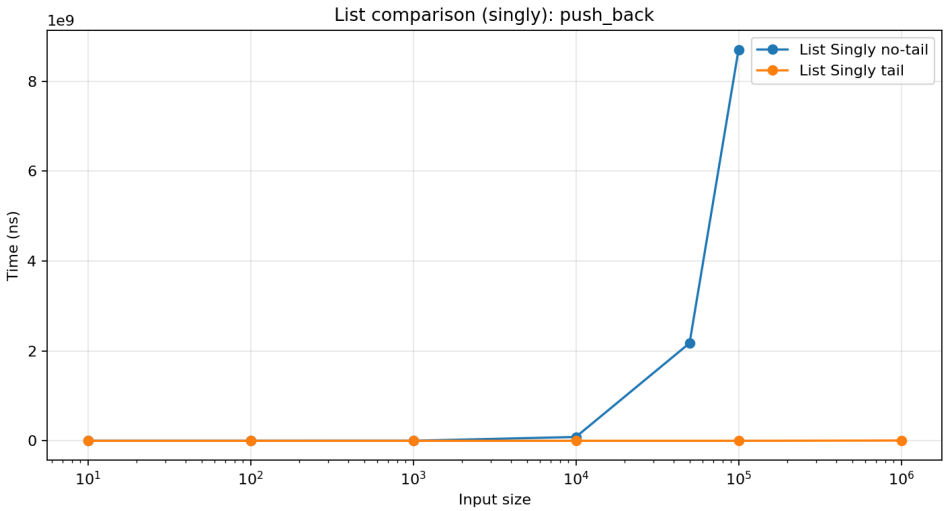

Figura 11: Comparación de `push_back` en listas simplemente enlazadas con y sin cola.

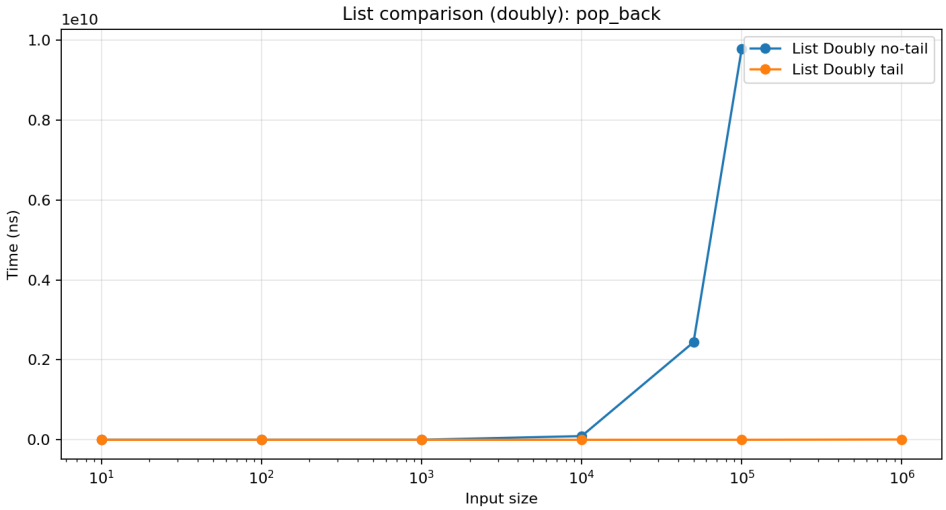

Figura 12: Comparación de `pop_back` en listas simplemente enlazadas con y sin cola.

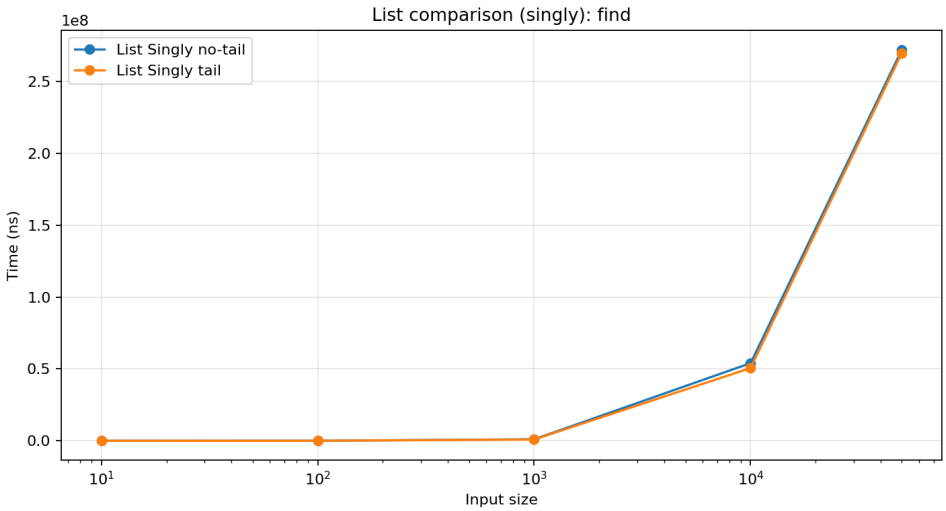

Figura 13: Comparación de `find` en listas simplemente enlazadas con y sin cola.

Comparación en listas simplemente enlazadas En las Figuras 11 y 12 se observa con claridad el impacto de incorporar una referencia `tail` en las listas simplemente enlazadas. En el caso de `push_back`, la versión con cola presenta una mejora importante frente a la versión sin cola, lo cual es consistente con el análisis teórico: al disponer de acceso directo al último nodo, deja de ser necesario recorrer toda la lista para insertar al final.

En `pop_back`, aunque la implementación con cola también mejora respecto a la versión sin cola, la diferencia no es tan drástica como en `push_back`. Esto se debe a que, aun teniendo una referencia al último nodo, en una lista simplemente enlazada sigue siendo necesario localizar el nodo anterior al final para poder actualizar correctamente los enlaces.

Por su parte, la Figura 13 muestra que la operación `find` mantiene un comportamiento muy similar en ambas implementaciones. Esto era esperable, ya que la presencia de `tail` no modifica el hecho de que la búsqueda deba realizarse recorriendo secuencialmente la estructura desde la cabeza.

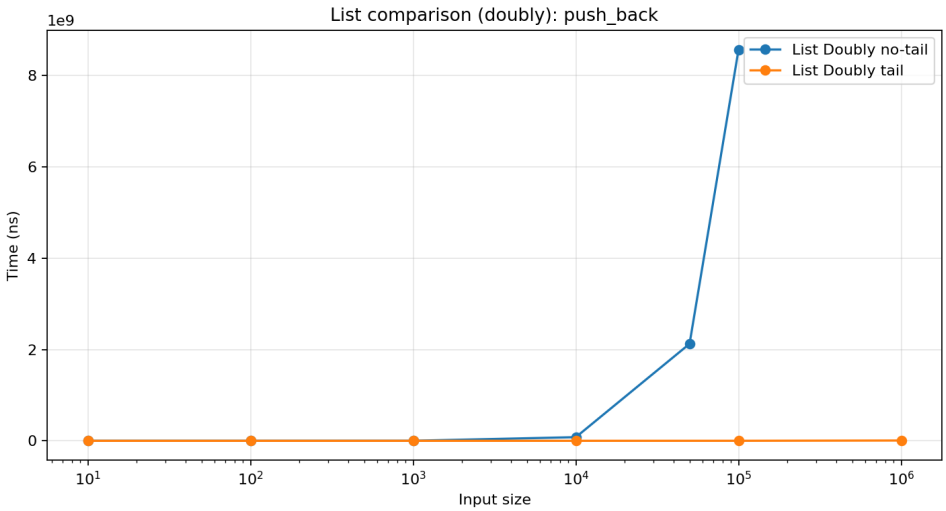

Figura 14: Comparación de `push_back` en listas doblemente enlazadas con y sin cola.

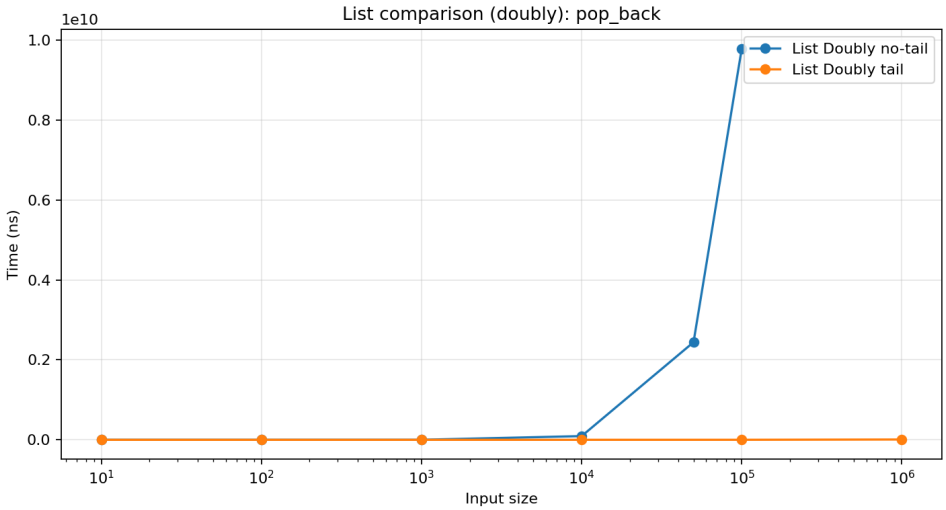

Figura 15: Comparación de `pop_back` en listas doblemente enlazadas con y sin cola.

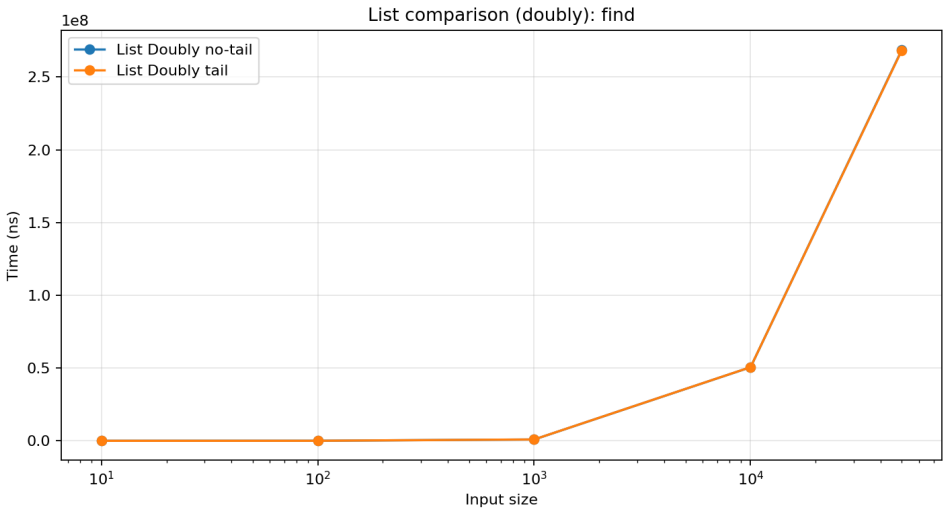

Figura 16: Comparación de `find` en listas doblemente enlazadas con y sin cola.

Comparación en listas doblemente enlazadas En las Figuras 14 y 15 se aprecia nuevamente que la referencia `tail` mejora de manera importante el comportamiento de las operaciones sobre el final de la lista. En `push_back`, el acceso directo al último nodo evita el recorrido completo de la estructura. En `pop_back`, además, la combinación de `tail` con referencias `prev` permite acceder inmediatamente al nodo anterior, lo que se traduce en una mejora aún más marcada que en el caso de las listas simplemente enlazadas.

En contraste, la Figura 16 evidencia que `find` mantiene un comportamiento similar en ambas variantes doblemente enlazadas. Nuevamente, esto confirma que la presencia de `tail` no altera la naturaleza secuencial del proceso de búsqueda.

Síntesis En conjunto, estas comparaciones muestran que la incorporación de una referencia `tail` tiene un efecto decisivo en operaciones relacionadas con el extremo final de la lista, particularmente en `push_back` y `pop_back`. Sin embargo, también se observa que dicha mejora no se extiende a todas las operaciones: en métodos como `find`, el comportamiento permanece prácticamente inalterado, ya que su costo depende del recorrido de la estructura y no del acceso al último nodo.

Este resultado refuerza una idea importante del análisis de estructuras de datos: ciertas decisiones de diseño pueden optimizar operaciones muy específicas sin modificar necesariamente el comportamiento global de toda la interfaz.

5.3.6. Comparación entre estructuras para operaciones equivalentes

Como parte final del análisis empírico, se comparan operaciones equivalentes entre `List`, `MyStack<T>` y `MyQueue<T>`. Siguiendo la directriz del proyecto, para este tipo de comparación se selecciona, en cada caso, la implementación de `List` que resulta más apropiada

desde el punto de vista teórico y experimental.

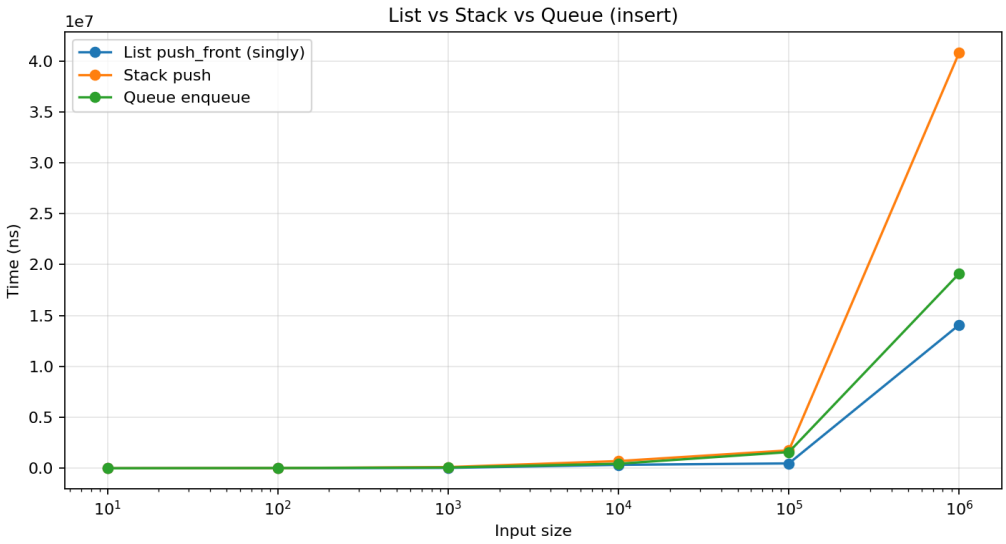

Figura 17: Comparación entre operaciones equivalentes de inserción en `List`, `MyStack<T>` y `MyQueue<T>`.

Inserción: `push_front` vs `push` vs `enqueue` En la Figura 17 se comparan tres operaciones de inserción: `push_front` en una lista simplemente enlazada, `push` en la pila y `enqueue` en la cola. Para representar a `List` se eligió la operación `push_front` sobre la implementación simplemente enlazada, ya que constituye una inserción natural y eficiente en una lista enlazada, sin depender de referencias adicionales como `tail` o `prev`. En otras palabras, se trata de una implementación suficientemente simple, pero al mismo tiempo óptima para este tipo de operación.

Desde el punto de vista teórico, las tres operaciones presentan costo constante por llamada. La comparación experimental resulta entonces útil para observar no solo la complejidad asintótica, sino también las diferencias prácticas asociadas al tipo de implementación utilizada en cada estructura.

La gráfica muestra que las tres curvas mantienen una tendencia similar a lo largo de los distintos tamaños de entrada, lo que confirma que las tres estructuras realizan inserciones eficientes en este contexto. Sin embargo, en tamaños grandes se observa que la inserción en la lista simplemente enlazada presenta los menores tiempos, seguida por `enqueue` en la cola, mientras que `push` en la pila exhibe una sobrecarga algo mayor.

Estas diferencias no implican un cambio en la complejidad teórica de las operaciones, sino que reflejan detalles de implementación y efectos del entorno de ejecución, como el manejo de arreglos dinámicos, el redimensionamiento y el patrón de acceso a memoria. En particular, `push_front` en la lista enlazada solo requiere crear un nodo y actualizar

la referencia de cabeza, mientras que las implementaciones de pila y cola dependen de arreglos dinámicos, lo cual introduce una sobrecarga adicional en tamaños elevados.

6. Conclusiones

A partir del análisis teórico y experimental realizado sobre las estructuras `List`, `MyStack<T>` y `MyQueue<T>`, es posible establecer conclusiones claras sobre su comportamiento, ventajas y escenarios de uso.

- **Selección de estructura según la operación dominante**

  Los resultados evidencian que no existe una única estructura óptima para todos los casos, sino que la elección depende directamente del tipo de operaciones que se realizan con mayor frecuencia. En particular:

  - Las operaciones sobre extremos (`push`, `pop`, `enqueue`, `dequeue`) presentan un comportamiento eficiente en pilas y colas, confirmando su idoneidad para escenarios donde el acceso se restringe a un extremo.
  - En listas enlazadas, operaciones como `push_front` y `pop_front` resultan eficientes, mientras que operaciones sobre el final (`push_back`, `pop_back`) dependen fuertemente de la implementación (presencia de `tail` o enlaces dobles).
  - Las operaciones que implican búsqueda (`find`, `erase`, `add_before`, `add_after`) presentan un crecimiento significativo con el tamaño de entrada, lo que confirma su costo lineal y su impacto en el desempeño general.

- **Impacto de las decisiones de implementación**

  El análisis comparativo mostró que pequeñas variaciones en la implementación pueden tener un impacto considerable en el desempeño:

  - La incorporación de una referencia `tail` mejora significativamente las operaciones en el extremo final de la lista, especialmente en `push_back`.
  - En listas doblemente enlazadas, la combinación de `tail` con referencias `prev` permite optimizar también `pop_back`, reduciendo su costo respecto a otras variantes.
  - Sin embargo, estas mejoras no afectan operaciones como `find`, donde el costo sigue dominado por el recorrido completo de la estructura.

- **Comportamiento práctico vs. complejidad teórica**

  Aunque múltiples operaciones comparten la misma complejidad asintótica, los resultados experimentales muestran diferencias en los tiempos reales de ejecución. Estas

diferencias se deben a factores como:

- el uso de arreglos dinámicos en pilas y colas,
- el costo de creación de nodos en listas enlazadas,
- y los patrones de acceso a memoria.

En particular, se observó que, para tamaños grandes, las implementaciones basadas en arreglos pueden presentar una ligera sobrecarga frente a operaciones equivalentes en listas enlazadas, aun cuando todas mantengan complejidad constante amortizada.

- **Escenarios de uso en aplicaciones reales**

  Los resultados obtenidos permiten asociar cada estructura con escenarios concretos:

  - Las **pilas** son adecuadas para problemas de tipo LIFO, como evaluación de expresiones, manejo de llamadas recursivas o deshacer operaciones.
  - Las **colas** son apropiadas para sistemas FIFO, como gestión de tareas, procesamiento de solicitudes o simulaciones de eventos.
  - Las **listas enlazadas** ofrecen mayor flexibilidad para inserciones y eliminaciones en posiciones arbitrarias, siendo útiles en estructuras dinámicas donde no se conoce el tamaño de antemano o donde se requiere modificar la estructura con frecuencia.

- **Arreglos dinámicos vs. listas enlazadas**

  Finalmente, el análisis permite contrastar ambas estrategias de implementación:

  - Los **arreglos dinámicos** ofrecen mejor localidad de memoria y acceso más rápido en operaciones simples, pero pueden incurrir en costos adicionales por redimensionamiento.
  - Las **listas enlazadas** permiten inserciones y eliminaciones más flexibles sin necesidad de reubicar elementos, aunque presentan mayor sobrecarga en memoria y menor eficiencia en acceso secuencial.

  En conjunto, la elección entre ambas alternativas depende del tipo de acceso requerido, la frecuencia de inserciones y eliminaciones, y el tamaño esperado de la estructura.

7. Repositorio

El repositorio del proyecto se encuentra disponible en el siguiente enlace:

Repositorio en GitHub